

Industrial Control
25791

Behzad Ahi **Assignment 1**

Due Date: 1405/02/10

Note 1: Bonus problems are indicated in ***purple***.
Note 2: You are expected to upload a PDF file that includes a *detailed derivation* procedure for each question. For the simulation part, along with the essential plots, it is necessary to *append* your *code* at the end of the PDF file. Additionally, provide the *source codes* in a separate folder, where the name of each source code corresponds to the number of the related question.

1 Z-Transform and Inverse Z-Transform

The z-transform of a discrete-time signal $x[n]$ is defined as:

$$X(z) = \sum_{n=-\infty}^{\infty} x[n] z^{-n}$$

We use Z-Transform to discritize systems via ZOH method.

a) Find the z-transform of the following signals:

1) $x_1[n] = (0.9)^n\, u[n] - (0.3)^n\, u[n]$, where $u[n]$ is the unit step function.
2) $x_3[n] = n a^n u[n]$
3) $x_4[n] = a^n \cos(\omega_0 n) u[n]$

b) Find the inverse z-transform of the following discrete-time transfer functions using partial fraction expansion:

1) $G_1(z) = \dfrac{1}{z(z-0.7)}$
2) $G_2(z) = \dfrac{z^2 + 2z + 1}{z^2 - 0.5z - 0.12}$
3) $G_3(z) = z^5$

c) Verify your answers in first part of `a` and `b` using MATLAB. You may use the following MATLAB functions:

I) `ztrans` to find the z-transform of a symbolic expression.
II) `iztrans` to find the inverse z-transform of a symbolic expression.

2 Stability of Discrete-Time Systems

The stability of a discrete-time transfer function is determined by the location of its poles in the z-plane. A discrete-time system is stable if and only if all its poles lie inside the unit circle. One method for checking stability is the Jury test.

In this question, we use an alternative approach. The bilinear transformation maps the inside of the unit circle in the z-plane to the left-half plane in the s-plane, making it possible to use the Routh–Hurwitz stability criterion on the transformed polynomial.

Consider the discrete-time transfer function

$$H(z) = \frac{1}{z^3 + 2.7z^2 + 2.8z + 1.5}$$

a) Use the bilinear transformation to map the characteristic polynomial from the z-plane to the s-plane. Then apply the Routh–Hurwitz criterion to determine whether the system is stable.

b) Check the stability of the system again by directly computing the roots of the characteristic polynomial and evaluating whether they lie inside the unit circle. You may use the MATLAB function `roots` to find the poles.

3 Discretization of Continuous-time Controllers

Consider following two continuous-time controllers

$$G(s) = 5 + \frac{12}{s+1}, \qquad H(s) = \frac{8(s+3)}{s^2 + 10s + 29}$$

a) For each controller, select an appropriate **sampling time**.

b) Use **pole-zero matching** technique to obtain the discritized version.

c) Use **Tustin transformation** to obtain the discritized version.

d) Use **bilinear transformation** with warping frequency at $\omega_0 = 5$ rad/s to obtain the discritized version.

e) Plot and compare **frequency responses** of discritized controllers with the original continuous one.

Hint: You can use Matlab `c2d` and `Bode` commands.

4 Choosing suitable Discretization Method

Discretization is the process of converting continuous functions, models, variables, and equations into discrete counterparts. This is often necessary for numerical evaluation and implementation on digital computers. In control systems, discretization involves transforming continuous differential equations into discrete difference equations, which can be solved numerically. The MATLAB function `c2d` is used to convert a continuous-time dynamic system model into a discrete-time model. The syntax for this function is `sysd = c2d(sysc,Ts,method)`, where `sysc` is the continuous system, `Ts` is the sample time, and `method` is the discretization method you choose. The choice of method depends on the specific requirements of your system and the type of input signals it will handle. For instance, if you're working with signals that change in a stepwise fashion, ZOH will be suits well. If the signal is more smooth or linear, FOH could be better. Tustin's method is often a good compromise between time and frequency domain characteristics. The possible `c2d` discritization options are provided in following table.

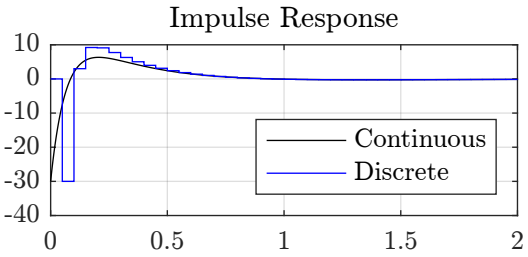
Zero-Order Hold (ZOH)	`c2d(sys, Ts, 'zoh');`
First-Order Hold (FOH)	`c2d(sys, Ts, 'foh');`
Impulse-Invariant Mapping	`c2d(sys, Ts, 'impulse');`
Tustin Approximation	`c2d(sys, Ts, 'tustin');`
Tustin + Prewarping	`c2d(sys, Ts, 'prewarp', omega);`
Zero-Pole Matching	`c2d(sys, Ts, 'matched');`
Least Squares	`c2d(sys, Ts, 'least-squares');`

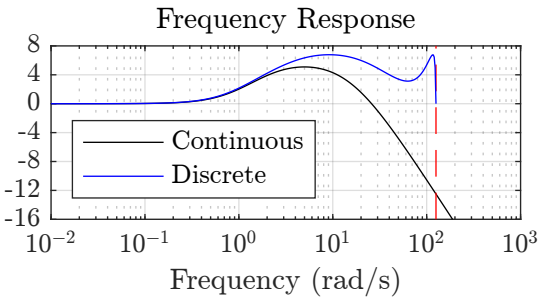
Discretization introduces an approximation, so there will be some level of discrepancies between continuous and discritized system. The goal is to minimize this error to an acceptable level for your application. Consider the following third-order system

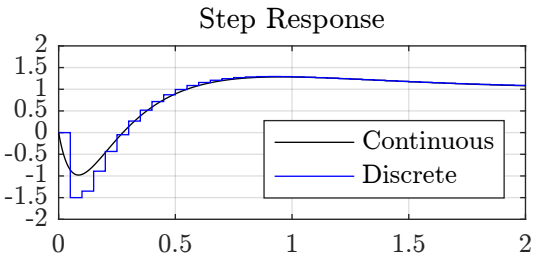
$$H(s) = \frac{-30s^2 + 60s + 90}{s^3 + 20s^2 + 81s + 90}$$

To gain an intuition, results from applying various discretization techniques to system $H(s)$ are depicted below (you might want to regenerate these results)

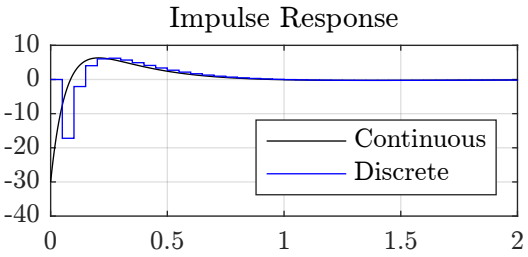
Forward Rectangular Rule

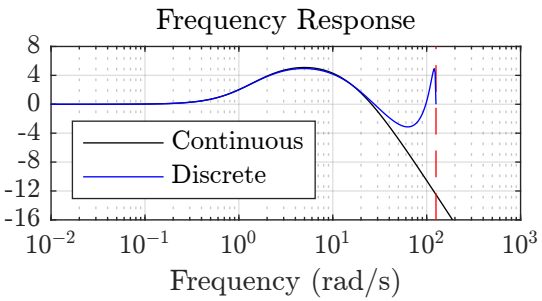


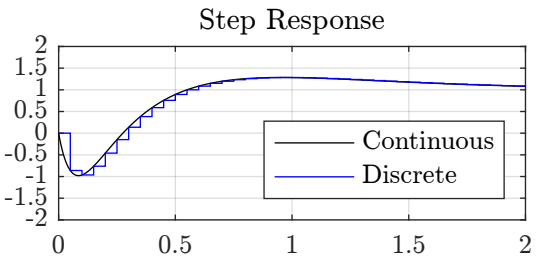




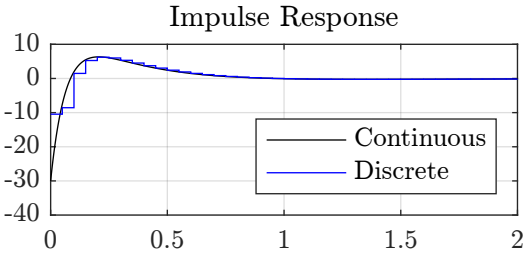
Zero-Order Hold (ZOH)

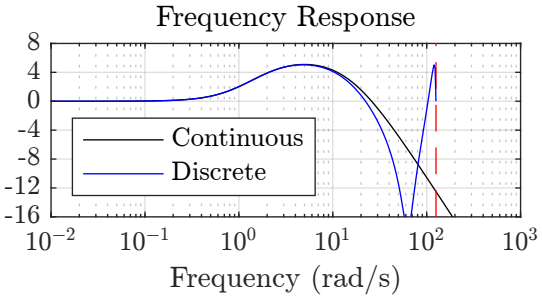


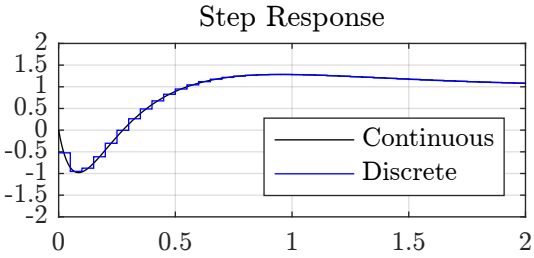




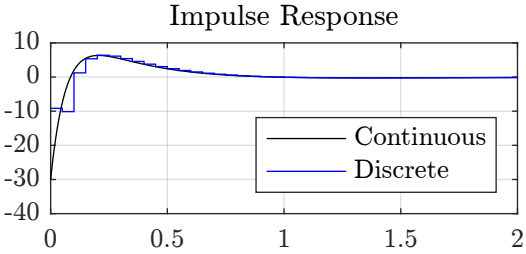
First-Order Hold (FOH)

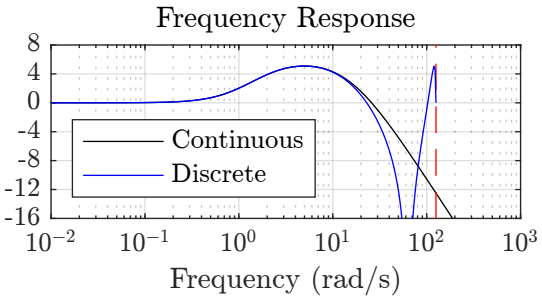


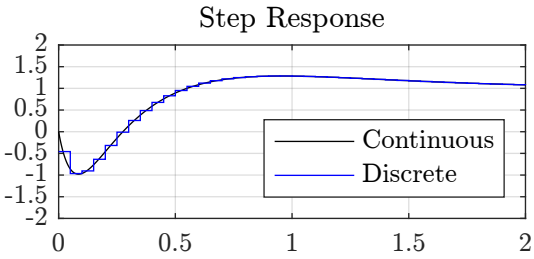




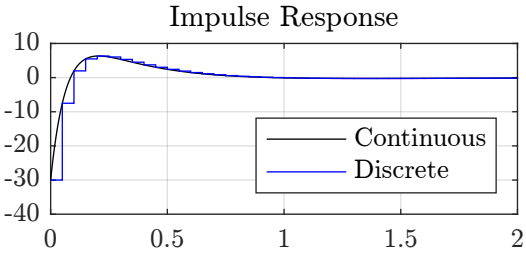
Tustin

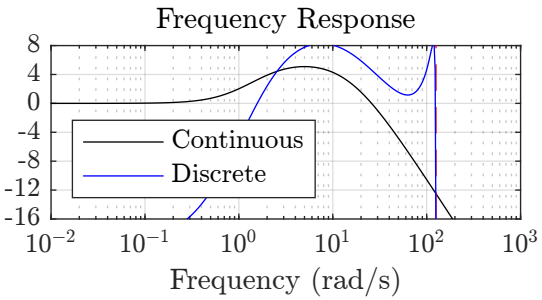


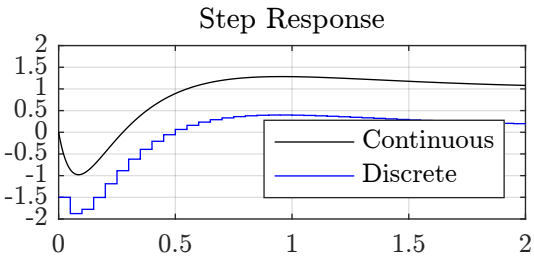




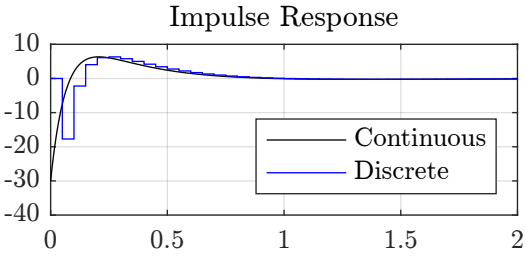
Impulse-Invariant

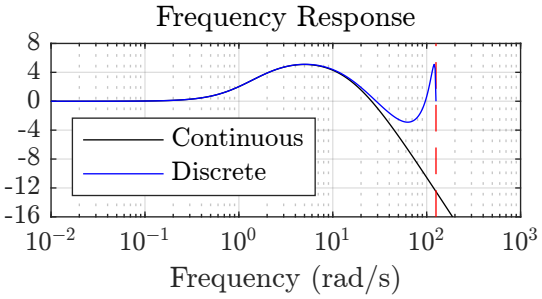


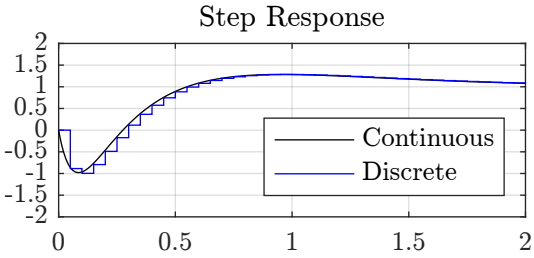




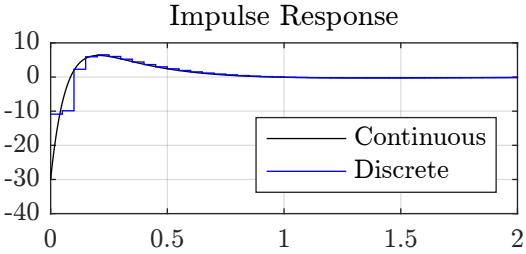
Zero-Pole Matching

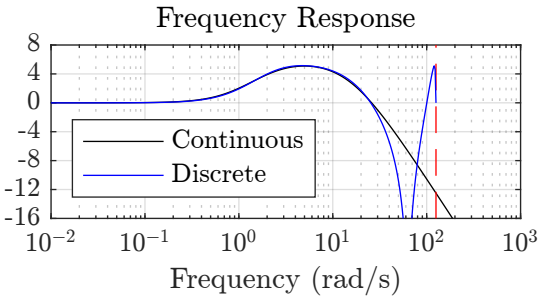


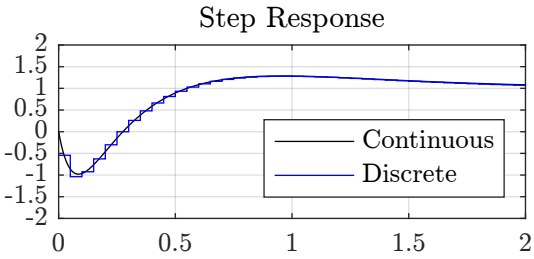




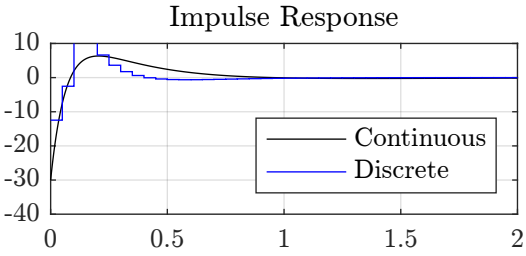
Least Squares

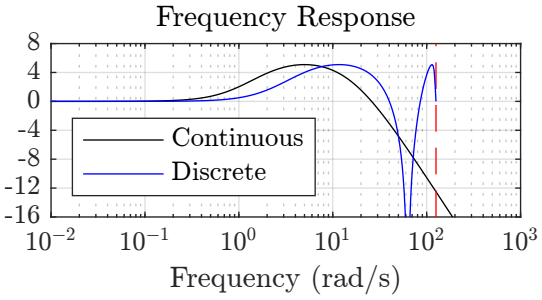


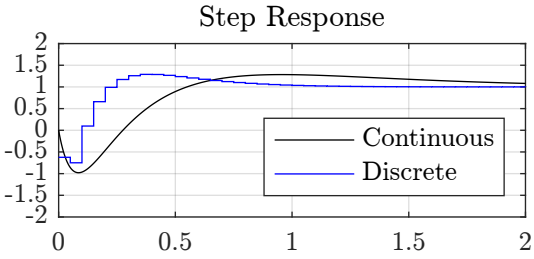




Tustin + Prewarping







Note: Prewarping frequency is 50 rad/s and it's too high for the given system!

Assignment: For each of the methods described, provide a brief explanation and outline the process of converting a continuous-time transfer function to a discrete-time transfer function using these methods. Subsequently, analyze the results to determine the most suitable method for discretizing *system* for design in the discrete-time domain and the preferred method for discretizing the designed *controller* in the continuous-time domain.
Plot the frequency responses of various methods in one Figure.

5 Automated Code Generation in `Simulink`

Modern control systems and embedded applications require efficient and reliable implementation of algorithms in programming languages such as C and C++. To facilitate this process, MathWorks' `Simulink` provides an advanced feature for automated code generation, allowing engineers to translate MATLAB scripts and `Simulink` models into optimized C, C++, CUDA, Verilog, or VHDL code. This feature significantly reduces manual coding efforts, enabling rapid prototyping, real-time testing, and deployment of control algorithms on embedded systems and hardware platforms.

Overview of `Simulink` Code Generation

`Simulink` offers a user-friendly environment for designing and simulating complex systems before deployment. The code generation process is streamlined through the **Embedded Coder** toolbox, which provides advanced optimization techniques such as execution speed enhancement, memory footprint reduction, and compliance with industry standards (such as MISRA-C). This functionality is particularly beneficial for control engineers working in industrial automation, robotics, signal processing, and real-time embedded applications.

To generate C code from a `Simulink` model, follow these general steps:

1. Open your `Simulink` model.
2. Navigate to **Embedded Coder** from the `Simulink` Apps.
3. Click on **Quick Start** and follow the guided instructions.
4. Ensure that the model uses a fixed-step solver to maintain discrete-time behavior.
5. If using subsystems, encapsulate the control logic within a dedicated `Simulink` Subsystem block for modular and reusable code.
6. Generate the C code and inspect the generated files, including the main source code, headers, and initialization routines.

Note: ***Variable step-size solvers should not be used****, as they prevent proper discretization of the system and can lead to issues in code generation.*

The objective of this assignment is to implement C code generation for a given discrete-time system using both `Simulink` and MATLAB's `.m` file approach. You are required to generate C code for the following system:

Discrete-Time State-Space Model

Consider a discrete-time state-space system represented by:

$$\mathbf{x}[k+1] = \begin{bmatrix} 0.8 & 0.3 \\ -0.2 & 0.7 \end{bmatrix} \mathbf{x}[k] + \begin{bmatrix} 0.2 \\ 0.1 \end{bmatrix} u[k]$$

$$y[k] = \begin{bmatrix} 1 & 0 \end{bmatrix} \mathbf{x}[k]$$

where:

- $x[k] \in R^2$ represents the state vector at time step k.
- $u[k]$ is the system input.
- $y[k]$ is the system output.
- The system operates with a sampling time of $T_s = 0.01$ seconds.

Assignment: You are required to complete the following tasks for the system:

a) **Generate C Code using `Simulink`.** Implement the given system in `Simulink`, configure the model for fixed-step simulation, and use the Embedded Coder to generate optimized C code.

b) **Generate C Code using a `.m` File.** Implement the same system in MATLAB script form and use the `codegen` function to generate the equivalent C code.

c) **Analysis of MATLAB's Implementation of the Integrator.** Explain how MATLAB translates the discrete-time integration process into C code. Provide insights into the numerical techniques used (e.g., Euler discretization, bilinear transform, etc.) and analyze the generated C functions for updating system states.

6 Optimization Toolbox

Optimization problems are encountered in various fields ranging from engineering to science and economics. Specifically, many real-word control problems can be formulated as an optimization problem. So, there exist many optimization-based control strategies such as those you will see in optimal control and model predictive control (MPC) courses.
The MATLAB Optimization Toolbox is a powerful collection of algorithms and tools designed to solve various optimization problems. It provides functions for finding the minimum or maximum of a function, subject to constraints, using a variety of optimization algorithms. Here's a comprehensive explanation of its key features and components:

I. Optimization Problem Types:
   i. Unconstrained optimization: Finding the minimum or maximum of a function without any constraints.
   ii. Constrained optimization: Optimizing a function subject to constraints, such as bounds on variables or equality and inequality constraints.
   iii. Linear programming (LP): Optimizing a linear objective function subject to linear equality and inequality constraints.
   iv. Quadratic programming (QP): Optimizing a quadratic objective function subject to linear equality and inequality constraints.
   v. Nonlinear programming (NLP): Optimizing a nonlinear objective function subject to nonlinear equality and inequality constraints.

II. Optimization Algorithms:
   i. Gradient-based methods: Such as steepest descent, conjugate gradient, and quasi-Newton methods like BFGS and L-BFGS.
   ii. Trust-region methods: Which build a quadratic model of the objective function and use it to guide the search.
   iii. Interior-point methods: Particularly useful for solving large-scale linear and quadratic programming problems.
   iv. Genetic algorithms: Evolutionary algorithms inspired by natural selection and genetics, useful for global optimization problems.

v. Simulated annealing: A probabilistic optimization technique inspired by the annealing process in metallurgy.

vi. Particle swarm optimization (PSO): An optimization technique inspired by the social behavior of birds flocking or fish schooling.

III. Problem Setup and Solution: Optimization problems are typically defined in MATLAB using functions or function handles representing the objective function and any constraints. The toolbox provides functions like `fmincon` for constrained nonlinear optimization, `linprog` for linear programming, `quadprog` for quadratic programming, etc., to solve specific types of optimization problems. Users can specify options and parameters for the optimization algorithms, such as convergence tolerances, maximum iterations, and display options.

IV. Visualization and Analysis: MATLAB offers visualization tools to analyze optimization results, including plotting functions, convergence plots, and diagnostic tools. Users can examine solution trajectories, convergence behavior, and sensitivity to parameter changes.

Assignment: We want to solve the following simple constrained optimization problem

$$\begin{array}{ll} \text{minimize} & f(x) = -2x_1x_2 - x_2x_3 \\ \text{subject to} & x_1^2 + x_2^2 + x_3^2 \leq 64 \\ & x_1, x_2, x_3 \geq 0 \end{array}$$

a) Use MATLAB's Optimization Toolbox to solve above problem (Hint: In recent versions of MATLAB you need to use optimization toolbox from the live script).

b) Use `fmincon`, `fminsearch` or other proper built-in functions to solve the given optimization problem.

7 Response of LTI State-Space Models using Euler Method

To find a numerical solution of an LTI state-space model using the Euler method iteratively in MATLAB, you need to follow these steps

I) Define the state-space model using the A, B, C, and D matrices

$$\text{System: } \begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases}$$

For example

```
A = [-1, 1; 0, -2];
B = [0; 1];
C = [1, 0];
D = 0;
```

II) Define the input signal `u` and the time vector `t`. The input signal `u` should be a matrix with as many rows as time samples and as many columns as system inputs. The time vector `t` should be a vector of regularly spaced time samples. For example, if you want to simulate the system for 10 seconds with a sampling time of 0.01 seconds and a step input, you can use

```
Ts = 0.01;
t = 0:0.01:10;
u = ones(size(t));
u(1:100) = 0;
```

III) Define a matrix to store the states samples `X` and specify the initial conditions

```
X = zeros([length(A), length(t)]);
x0 = [0; 0];
X(:, 1) = x0;
```

IV) Define a matrix to store the output samples `Y`

```
Y = zeros(size(t));
Y(1) = C * x0;
```

V) Use the Euler's formula to approximate the system response to the input signal. The Euler's formula is

$$x[i+1] = x[i] + T_s \dot{x}[i]$$

where

$$\dot{x}[i] = Ax[i] + Bu[i]$$

So

$$x[i+1] = x[i] + T_s \left(Ax[i] + Bu[i]\right)$$

VI) Implement the Euler's formula using a `for` loop

```
for i=2:length(t)
     X(:, i) = X(:, i-1) + Ts * (A * X(:, i-1) + B * u(i-1));
     Y(i) = C * X(:, i) + D * u(i);
end
```

Assignment: Given a linear time-invariant (LTI) state-space model and an input signal $u(t)$, use the Euler method to compute the state trajectories $x_1(t), x_2(t), \cdots$ and the output response $y(t)$ for the time interval $0 \leq t \leq 50$. Show the plots of the state and output variables on the same figure to illustrate how each state affects the system output.

$$\begin{cases} \dot{x}(t) = \begin{bmatrix} -2 & 1 & 0 & 0 & 0 \\ 0 & -1.2 & 0.5 & 0 & 0 \\ 0 & 0 & -3 & 0 & 0.8 \\ 0 & 0 & 0 & -1.5 & 0 \\ 0 & 0 & 0 & -0.6 & -4 \end{bmatrix} x(t) + \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} u(t) \\ y(t) = \begin{bmatrix} 1 & 0.5 & 1 & 0 & 0.2 \end{bmatrix} x(t) \end{cases}$$

$$u(t) = \begin{cases} 2 & 5 \leq t \leq 15 \\ -1 & 25 \leq t \leq 35 \\ 0 & O.W. \end{cases}$$

a) $x(0) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \end{bmatrix}^T$

b) $x(0) = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \end{bmatrix}^T$

c) $x(0) = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \end{bmatrix}^T$

8 Linearization

In practice, we usually face non-linear systems. It is hard to work with a nonlinear model since we require specific and complicated tools. If the perturbation around the operational point is small, we may obtain a linear model based on the actual model. It can be helpful in many cases. Here we will represent a brief review of the procedure before delving into the question. Assume that x denotes the state vector and u denotes the input vector. For the nonlinear system, let consider the following noise-free expression:

$$\dot{x} = F(x, u)$$
$$y = G(x, u)$$

where F and G are vectors of scalar functions. A stationary point (x_s, u_s) is a point in which $F(x_s, u_s) = 0$. These points are the ones around which we want to linearize the system. Let us define:

$$x = x_s + x_l$$
$$u = u_s + u_l$$

Then, we can find the linearized state-space representation of the system around (x_s, u_s) as follows:

$$\dot{x_l} = D_x F(x = x_s, u = u_s)x_l + D_u F(x = x_s, u = u_s)u_l$$
$$y = D_x G(x = x_s, u = u_s)x_l + D_u G(x = x_s, u = u_s)u_l$$

where $D_x F(x = x_s, u = u_s)$ is the Jacobian matrix of $F(x, u)$ with respect to x, evaluated at point (x_s, u_s).

Assignment: Consider the following system:

$$\dot{x} = F(x, u) = \begin{bmatrix} x_2 \\ x_1 - 0.2x_2 + x_1^3 + u \end{bmatrix}$$

$$y = x_1 + 0.1x_2^2 + 0.2u$$

around the following operating point:

$$x_s = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad u_s = -2$$

a) Compute the Jacobian matrices $A = D_x F(x_s, u_s)$, $B = D_u F(x_s, u_s)$, $C = D_x G(x_s, u_s)$, $D = D_u G(x_s, u_s)$, and write the linearized state-space model

$$\dot{x_l} = Ax_l + Bu_l, \qquad y_l = Cx_l + Du_l.$$

b) Check the stability of the obtained linearized system by computing the eigenvalues of A. Is the equilibrium point locally stable?

c) Design a proportional controller

$$u_l = -Kx_l$$

for the linearized model such that the closed-loop system

$$\dot{x}_l = (A - BK)x_l$$

becomes stable.

Choose a gain $K > 0$ ($K = [K_1 \quad K_2]$)that places both closed-loop eigenvalues in the left-half plane. You may use MATLAB's `eig` function to check how the eigenvalues move as you vary K.

d) Simulate the closed-loop linearized system for a small initial perturbation (e.g., $x_l(0) = [0.1\ 0]^T$) and show whether the controlled system converges back to the equilibrium.

9 Analyzing Linearized Model Performance Using Simulation

In this question, we evaluate how well the linearized model approximates the behavior of the original nonlinear system. Recall the perturbation definitions:

$$x_l = x - x_s, \qquad u_l = u - u_s, \qquad y_l = y - y_s.$$

The operating input u_s is constant.

a) Compare the responses of the nonlinear and linearized systems under a unity step change in input. To make the results directly comparable in a *single figure*, all responses must be expressed in the perturbation output form.

Apply the unity step as follows:

- Nonlinear system: apply

$$u(t) = u_s + 0.2.$$

Record the corresponding perturbation output

$$y_l^{\mathrm{nl}}(t) = y^{\mathrm{nl}}(t) - y_s.$$

- Linearized system: apply

$$u_l(t) = 0.2,$$

and record $y_l(t)$.

Plot all perturbation outputs $y_l(t)$ and $y_l^{\mathrm{nl}}(t)$ in a single figure and comment on how closely the linear model matches the nonlinear system near the operating point.

b) Compare whether linearization around $x_s = [-1, 0]$ gives a better approximation than the previous working point.

c) Examine how the accuracy of the linear model changes with larger input deviations. Repeat the simulations for larger perturbations (e.g.,$u_s + 1$,$u_s + 3$,$u_s + 5$). Convert all outputs to perturbation form and plot them together to assess increasing deviation.

10 PID Tuning using Empirical Methods

Tuning a PID (Proportional-Integral-Derivative) controller using empirical methods typically involves trial-and-error adjustments based on the system's response to different parameter settings. Here's a step-by-step approach to tune a PID controller empirically:

I. Start with a good initial guess: Use rule-of-thumb or manufacturer recommendations to set initial values for the proportional (K_p), integral (K_i), and derivative (K_d) gains. For example, you might start with $K_p = 1$, $K_i = 0$, and $K_d = 0$.

II. Test the system response to a step input: Plot the step response of the closed-loop system for the linear systems or apply a step change in the setpoint or disturbance to the system and observe its response. This could be done by manually changing the setpoint or using a test script if you have a simulation.

III. Assess the system's response: Observe the response of the system to the step input. Look for overshoot, settling time, and steady-state error. These parameters will give you insights into whether the controller is underdamped, overdamped, or critically damped, and whether there's any steady-state error.

IV. Adjust the proportional gain (K_p):

- If there is steady-state error, increase K_p to reduce it.
- If there is oscillation, instability or too much overshoot, decrease K_p.

V. Adjust the integral gain (K_i):

- If there's a constant error from the reference signal or it takes too much to reach a zero steady-state error, increase K_i.
- Be cautious not to increase K_i too much as it can lead to instability or slow response.

VI. Adjust the derivative gain (K_d):

- If there is overshoot or oscillation, increase K_d to dampen it.
- Be cautious with K_d as it can amplify noise and cause instability.

VII. Iterate: After adjusting the gains, repeat steps II→VI until you achieve satisfactory performance.

VIII. Fine-tuning: Once you're close to the desired response, fine-tune the gains to optimize performance. This might involve small adjustments to each gain to achieve the desired balance between speed of response, stability, and robustness to disturbances.

IX. Testing under different conditions: Test the tuned controller under various conditions, including different setpoints, disturbances, and operating conditions, to ensure robust performance across the entire operating range.

Typical effect of increasing the PID parameters on the step response characteristics of the closed-loop system is shown below:

	Rise Time	Overshoot	Settling Time	Steady-State Error
$\uparrow K_p$	$\downarrow$	$\uparrow$	$\uparrow$	$\downarrow$
$\uparrow K_i$	$\downarrow$	$\uparrow$	$\uparrow$	$\downarrow\downarrow\downarrow$
$\uparrow K_d$	$\uparrow$	$\downarrow$	$\downarrow$	$\times$

It is important to note that empirical tuning can be time-consuming and may not always yield an optimal solution. Therefore, one may be tempted to use optimization algorithms (such as MATLAB's `ga` function) to systematically determine suitable controller parameters.

Assignment: For each of the given systems, implement the control system in the Simulink and devise a suitable PID controller (reduce rise time, settling time and overshoot and eliminate the steady-state error of the closed-loop system) employing empirical methods. Using a `saturation` block, ensure that the input signal remains within the range of $-2 \leq u(t) \leq 2$. Plot both the reference and output signals on identical axes. Subsequently, establish the final recommendations for the PID parameters.

a) $G_1(s) = \dfrac{1}{(s+2)(0.15s+1)(0.05s+1)(0.01s+1)}$

b) $G_2(s) = \dfrac{1-1.5s}{(s+0.9)^3}$

c) $G_3(s) = \dfrac{1}{(3s+1)^2}e^{-0.6s}$

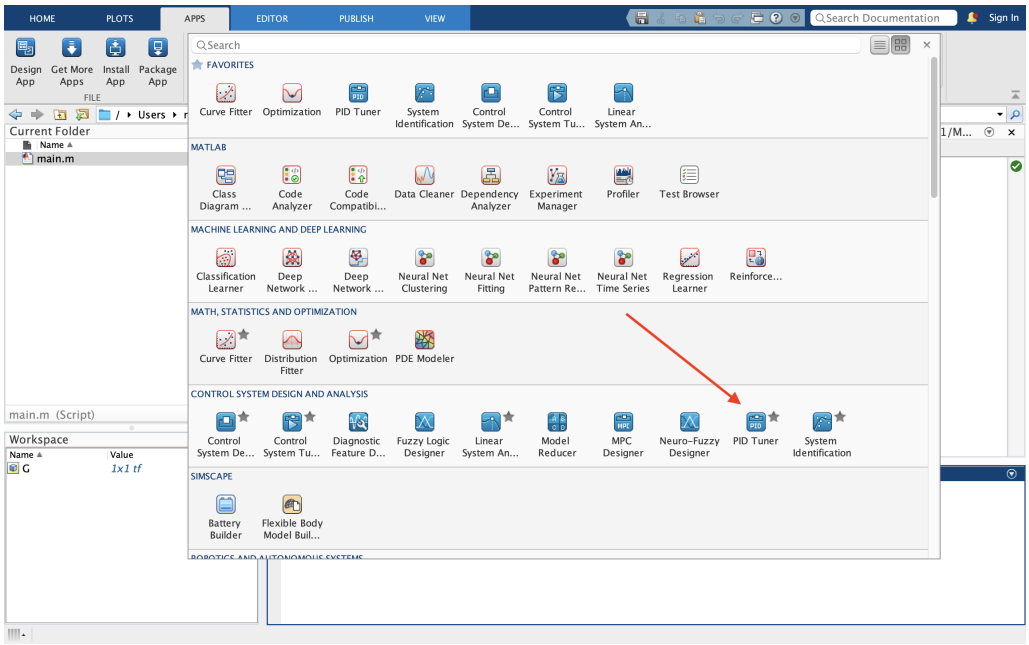
d) $G_4(s) = \dfrac{80}{(s+8)^2}\left(\dfrac{1}{s+2} + \dfrac{0.4}{s+0.04}\right)$

e) $G_5(s) = \dfrac{(s+5)^2}{s(s+2)^2(s+25)}$

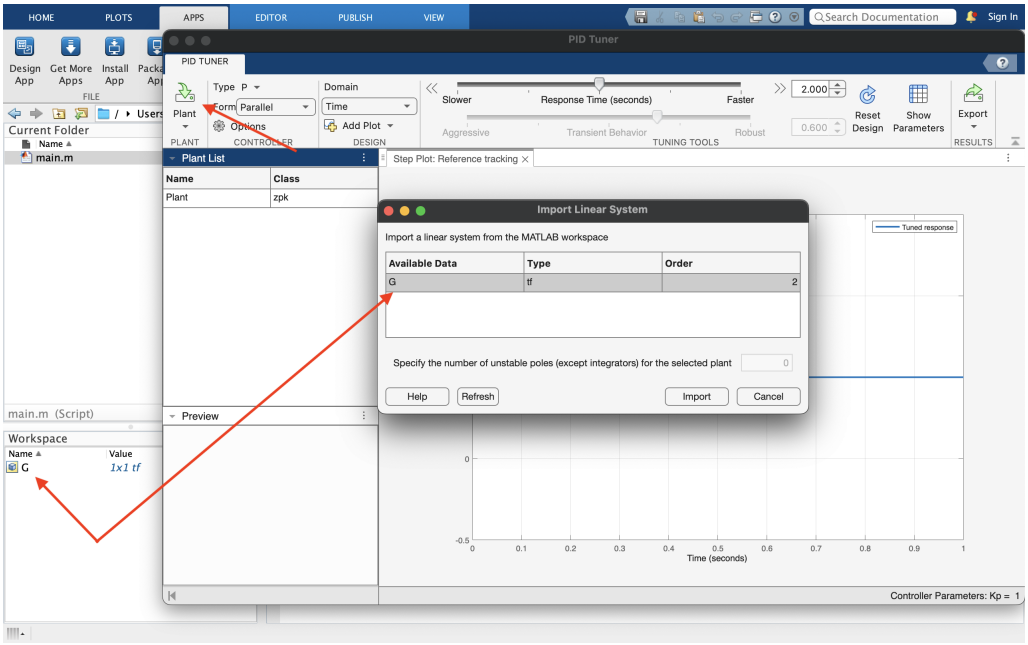
11 PID Tuning using PID Tuner App

PID tuning using the PID Tuner App in MATLAB is a process to design and optimize the parameters of a PID controller for a given system. The PID Tuner App provides an interactive graphical interface to adjust the controller parameters and observe the system response in real-time. Here's a step-by-step explanation of how to use the PID Tuner App:

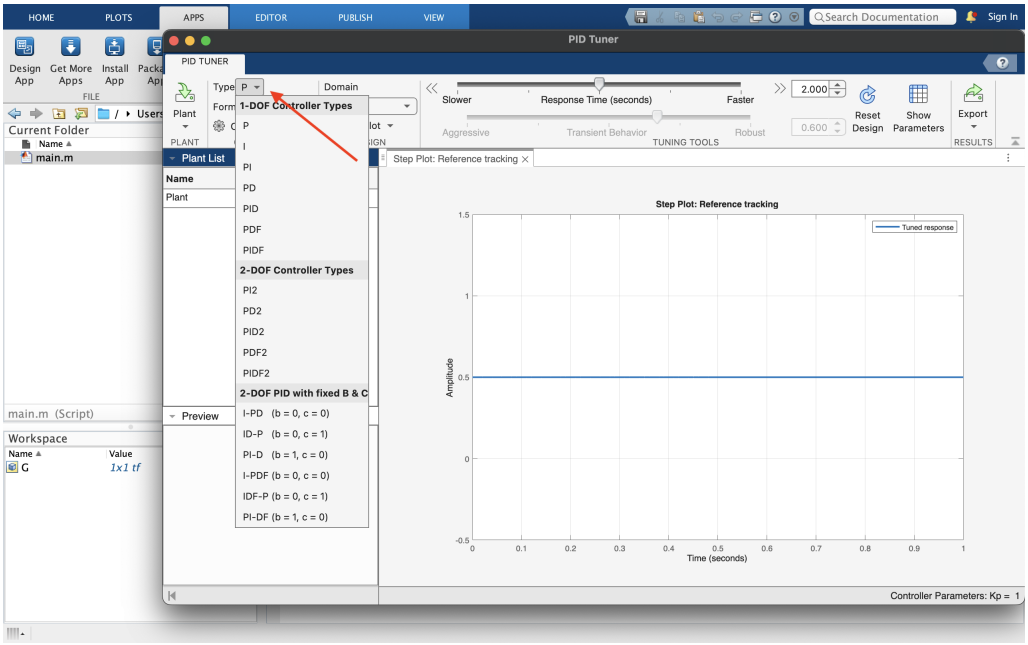
I. Open PID Tuner App: Launch MATLAB and open the PID Tuner App by typing `pidTuner` in the command window and hitting Enter. You can also open the PID Tuner App using applications window.



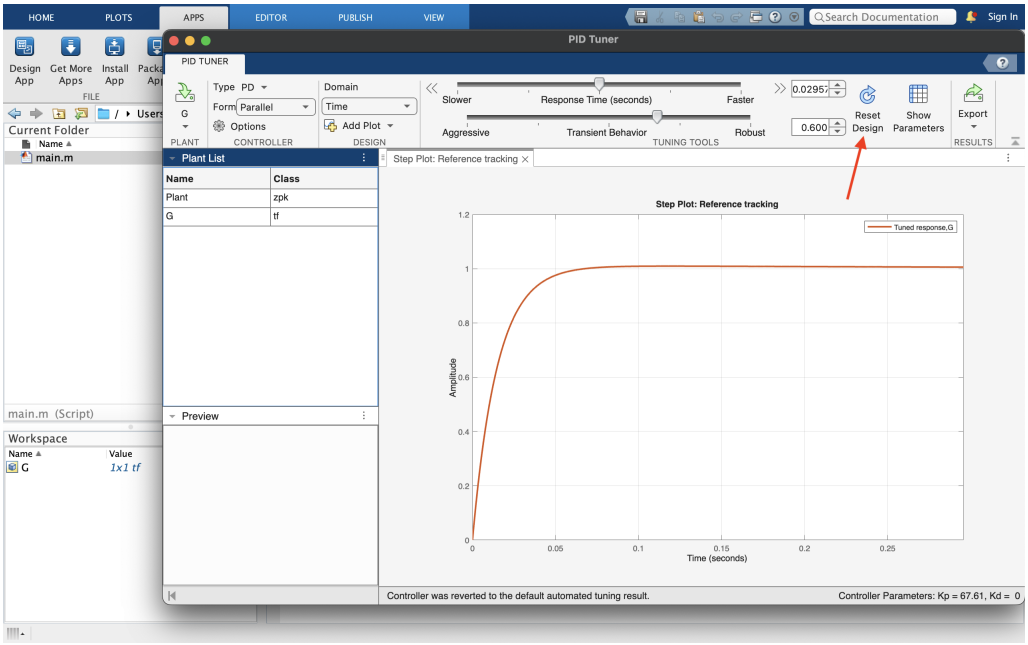
II. Select or Import Plant Model: Before tuning the PID controller, you need to have a plant model representing the system you want to control. You can either select an existing plant model or import one into the PID Tuner App. If you don't have a model, you can create a simple one using MATLAB's Control System Toolbox or use a Simulink model.



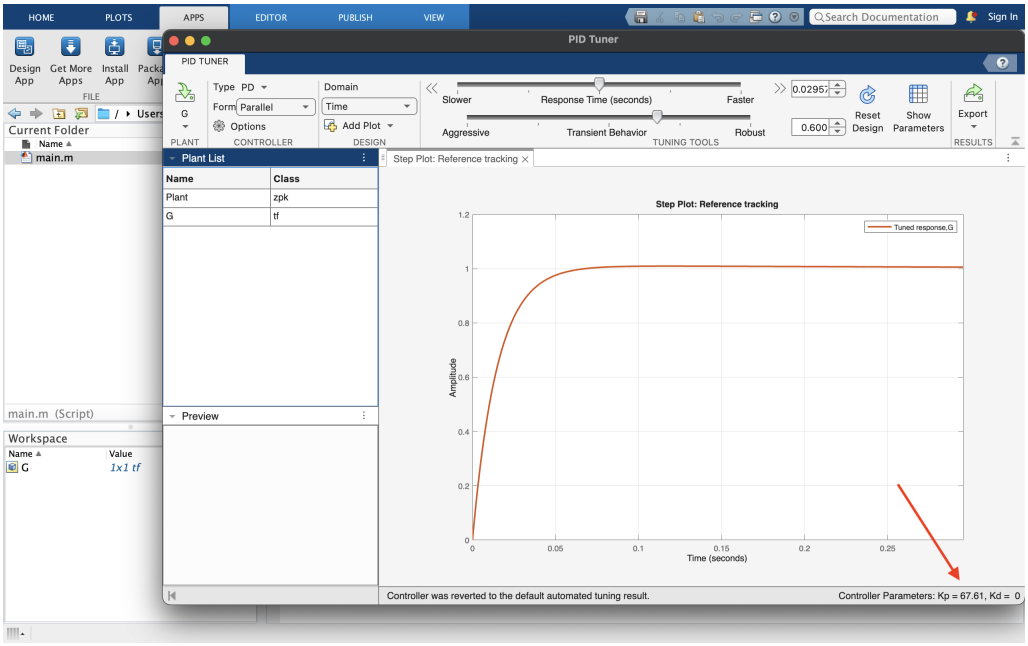
III. Select Controller Type: Choose the type of controller you want to tune. PID Tuner App supports tuning of PID controllers as well as PID with derivative filter (PD), PID with lead (PI) controllers.



IV. Initial Tuning: The PID Tuner App will initially set some default values for the PID controller parameters (Proportional, Integral, and Derivative gains). These initial values are based on some heuristics and may not be optimal for your system.

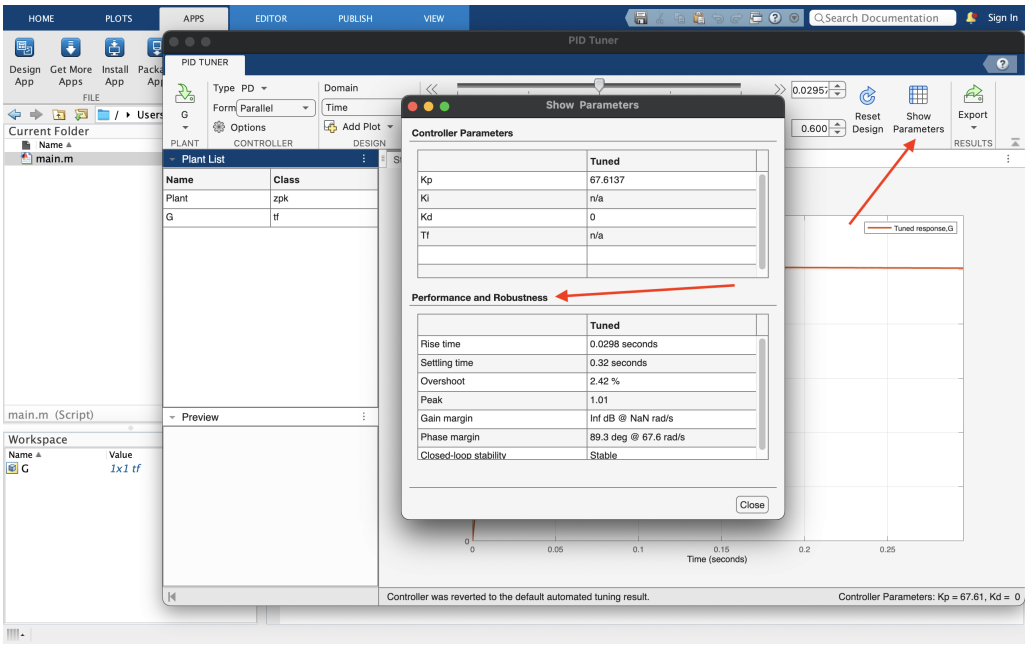


V. Adjust Controller Parameters: Use the sliders or input boxes in the PID Tuner App to adjust the PID controller parameters (Kp, Ki, Kd) to improve the system's performance. You can also adjust the filter settings if you're using a PID with a derivative filter or lead controller.



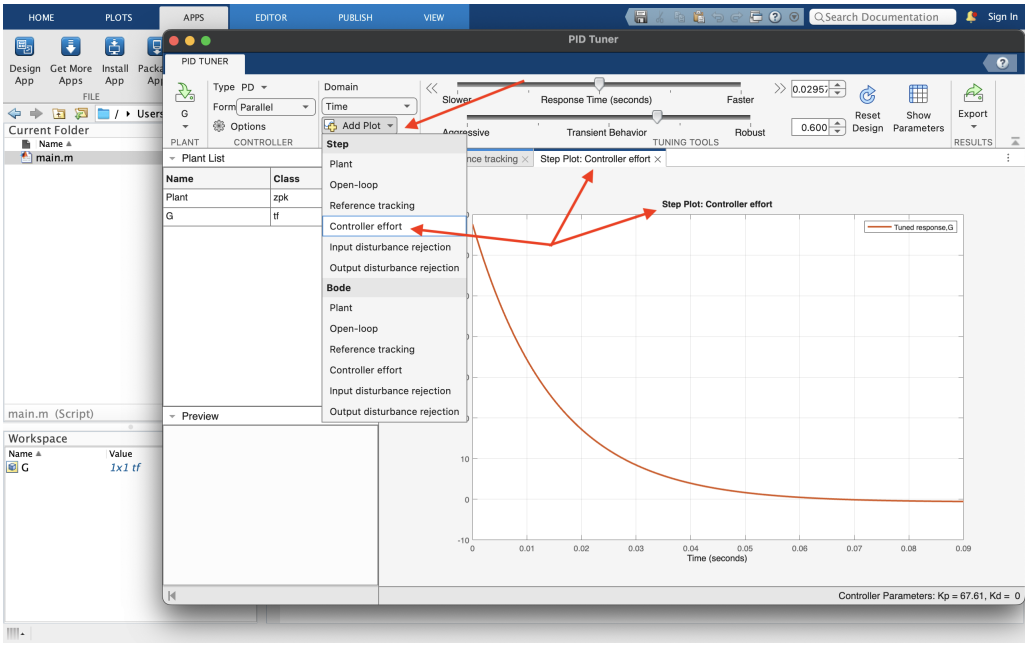
VI. Performance Metrics: While adjusting the parameters, keep an eye on the performance metrics displayed in the PID Tuner App. These metrics include rise time, settling time, overshoot, etc.

The goal is to minimize these metrics to achieve the desired system response. You should check the input signal to ensure it's implementable and feasible.

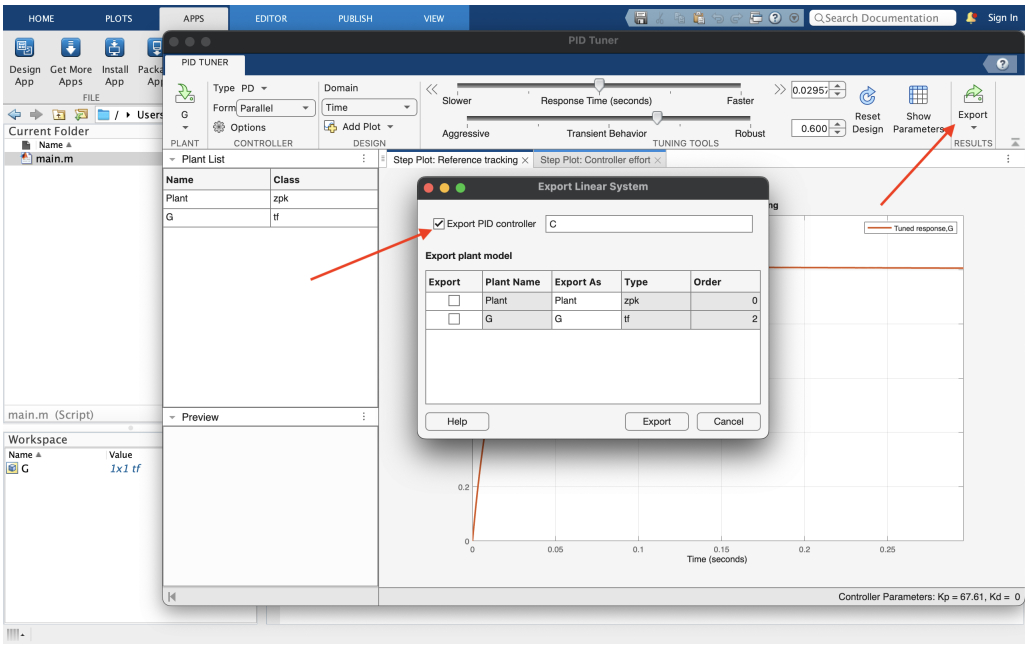


VII. Iterative Tuning: Iteratively adjust the controller parameters and observe the system response. Make small changes to the parameters and observe how they affect the system's behavior. Continue this process until you are satisfied with the performance of the system and make sure the input signal is still in the feasible domain.

VIII. Validation: Once you have tuned the PID controller parameters, validate the performance of the system by simulating it with the tuned controller. The PID Tuner App provides options to simulate the closed-loop system and analyze its response. You can also check other important plots of the control system. For example you can plot the input signal $u(t)$ like this:



IX. Export Controller: Once you're satisfied with the tuned PID controller parameters, you can export the controller to your MATLAB workspace for further analysis or implementation in your control system.



Assignment: Use PID Tuner App to design a PID controller for the system given in `part a) of Question 10`.